



TRABALHO PRÁTICO ESTRUTURAS DE DADOS E ALGORITMOS

Sistema de Indexação, Pesquisa e Análise de Dados com Tabela Hash

Linguagem: Java

Estrutura: Hash Table

Cotação: 20 valores

Dados: estudantes.csv

Nome(s):

Turma:

Data:

1. Contextualização

Uma instituição de ensino possui milhares de registos de estudantes. Cada estudante é identificado por um **código único** e possui informações académicas associadas.

À medida que a quantidade de registos aumenta, efectuar pesquisas através de um percurso sequencial torna-se menos eficiente.

Objectivo do trabalho

Desenvolver, em Java, um sistema que leia os dados de um ficheiro CSV e os armazene numa **Tabela Hash implementada pelo estudante**, permitindo realizar pesquisas, inserções, remoções, estatísticas e experiências de desempenho.

O trabalho deverá integrar:

- ❖ leitura e validação de dados CSV;
- ❖ implementação própria de uma Tabela Hash;
- ❖ Double Hashing para tratamento de colisões;
- ❖ pesquisa pelo código do estudante;
- ❖ inserção e remoção;
- ❖ redimensionamento automático;
- ❖ produção de estatísticas;
- ❖ avaliação experimental do desempenho.

2. Ficheiro de Dados

Será disponibilizado o ficheiro:

estudantes.csv
10 000 registos

O ficheiro possui a seguinte estrutura:

Campo	Descrição	Tipo sugerido
codigo	Código único do estudante	String
nome	Nome completo	String
curso	Curso frequentado	String
ano	Ano frequentado	int
idade	Idade do estudante	int
sexo	Sexo do estudante	String
media	Média académica	double

Exemplo:

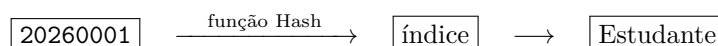
```
codigo,nome,curso,ano,idade,sexo,media
20260001,Ana Manuel,Engenharia Informatica,2,20,F,14.5
20260002,Carlos Jose,Engenharia Telecomunicacoes,3,22,M,12.8
20260003,Marta Paulo,Engenharia Informatica,1,19,F,16.2
```

3. Chave da Tabela Hash

Atenção

O campo `codigo` constitui a **chave da Tabela Hash**.
Cada código deverá ser único e será utilizado nas operações de **inserção, pesquisa e remoção**.

Por exemplo:



Para o registo:

Código	Nome	Curso	Ano	Idade	Sexo	Média
20260001	Ana Manuel	Eng. Informática	2	20	F	14.5

a chave é:

20260001

e o valor associado é o objecto contendo todos os dados da estudante.

4. Representação da Tabela Hash

A Tabela Hash deverá ser representada através de um **array unidimensional**.

Exemplo:

```
Entrada[] tabela = new Entrada[capacidade];
```

Cada posição do array deverá representar uma entrada da tabela.

Uma possível estrutura é:

```
class Entrada {
    Estudante estudante;
    int estado;
}
```

e:

```
class Estudante {
    String codigo;
    String nome;
    String curso;
    int ano;
    int idade;
    String sexo;
    double media;
}
```

Visualização conceptual

Índice	Estado	Código	Estudante
0	VAZIO	–	–
1	OCUPADO	20260003	Marta Paulo
2	REMOVIDO	–	–
3	OCUPADO	20260001	Ana Manuel
4	OCUPADO	20260002	Carlos José
5	VAZIO	–	–

Nota: esta tabela é apenas uma representação visual. Na implementação, trata-se de um **array unidimensional de entradas**.

5. Leitura do CSV

O programa deverá:

1. abrir o ficheiro `estudantes.csv`;
2. ignorar o cabeçalho;
3. ler os registos linha por linha;
4. separar os campos;
5. converter os valores para os tipos adequados;
6. validar os dados;
7. criar um objecto `Estudante`;
8. inserir o estudante na Tabela Hash;
9. contabilizar registos inválidos.

A leitura deverá permitir processar os **10 000 registos** disponibilizados.

6. Implementação da Tabela Hash

A Tabela Hash deverá ser implementada pelos estudantes utilizando **endereçamento aberto com Double Hashing**.

A sequência de sondagem será:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

onde:

k	chave (código do estudante);
i	número da tentativa;
m	capacidade da tabela;
$h_1(k)$	função Hash principal;
$h_2(k)$	passo da sondagem.

As funções h_1 e h_2 deverão ser implementadas e explicadas no relatório.

Estruturas proibidas

Não é permitido utilizar como estrutura principal:

HashMap Hashtable HashSet LinkedHashMap TreeMap

A finalidade do trabalho é implementar a própria Tabela Hash.

7. Estados das Posições

Cada posição deverá possuir um dos seguintes estados:

VAZIO

OCUPADO

REMOVIDO

Ao remover um estudante, a posição **não deverá ser simplesmente marcada como VAZIO**. O estado **REMOVIDO** permite que a sequência de sondagem continue durante pesquisas posteriores.

8. Operações Obrigatórias

A implementação deverá disponibilizar, no mínimo:

```
void inserir(Estudante estudante);

Estudante pesquisar(String codigo);

boolean remover(String codigo);
```

O sistema deverá permitir:

- ❖ pesquisar estudante pelo código;
- ❖ visualizar os dados encontrados;
- ❖ inserir um novo estudante;
- ❖ remover um estudante;
- ❖ visualizar a quantidade de estudantes;
- ❖ apresentar estatísticas dos estudantes;
- ❖ apresentar estatísticas da Tabela Hash.

Deverão ser tratados correctamente:

- ❖ códigos inexistentes;
- ❖ códigos duplicados;
- ❖ colisões;
- ❖ posições removidas;
- ❖ redimensionamento da tabela.

9. Factor de Carga e Redimensionamento

O factor de carga deverá ser calculado por:

$$\alpha = \frac{n}{m}$$

onde n representa o número de posições ocupadas e m a capacidade da tabela.

Quando:

$$\alpha > 0,70$$

a tabela deverá ser automaticamente redimensionada.

Redimensionamento

Os elementos existentes deverão ser novamente inseridos através das funções Hash.

Não é permitido copiar directamente os elementos das mesmas posições do array antigo para o novo.

10. Estatísticas dos Dados

Após a leitura do CSV, o programa deverá produzir automaticamente um relatório estatístico.

10.1 Estatísticas gerais

- ❖ total de linhas lidas;
- ❖ registos válidos;
- ❖ registos inválidos;
- ❖ estudantes inseridos.

10.2 Estatísticas por sexo

Apresentar:

- ❖ quantidade masculina;
- ❖ quantidade feminina;
- ❖ respectivas percentagens.

10.3 Estatísticas por curso

Para cada curso:

- ❖ quantidade de estudantes;
- ❖ percentagem relativamente ao total;
- ❖ média académica.

Exemplo:

```
Engenharia Informatica
-----
Estudantes : 4250
Percentagem : 42.5%
Media : 13.8
```

10.4 Estatísticas por ano

Apresentar o número de estudantes do:

10.5 Estatísticas das médias

Calcular:

- ❖ média académica geral;
- ❖ maior média;
- ❖ menor média;
- ❖ estudante com maior média;
- ❖ estudante com menor média;
- ❖ quantidade com média inferior a 10;
- ❖ quantidade entre 10 e 13,9;
- ❖ quantidade entre 14 e 16,9;
- ❖ quantidade com média igual ou superior a 17.

10.6 Estatísticas de idade

Calcular:

- ❖ idade média;
- ❖ menor idade;
- ❖ maior idade;
- ❖ quantidade por intervalo etário.

Intervalos:

< 18 18–20 21–23 24–26 > 26

11. Estatísticas da Tabela Hash

O sistema deverá também apresentar métricas sobre o comportamento da própria estrutura.

Métrica	Obrigatória
Capacidade inicial	Sim
Capacidade final	Sim
Elementos armazenados	Sim
Factor de carga final	Sim
Total de colisões	Sim
Total de sondagens	Sim
Média de sondagens por inserção	Sim
Maior número de sondagens	Sim
Número de redimensionamentos	Sim

Exemplo:

```
===== ESTATISTICAS DA HASH TABLE =====

Capacidade inicial : 1009
Capacidade final : 20011
Elementos armazenados : 10000
Factor de carga : 0.4997
Colisoes : 4287
Sondagens realizadas : 14287
Media sondagens/insercao : 1.43
Maior sondagem : 12
Redimensionamentos : 4
```

12. Experiência de Desempenho

Deverão ser realizados testes com:

1 000

5 000

10 000

registos.

Preencher a seguinte tabela:

Registos	Colisões	Média sond.	Máx. sond.	Carga	Tempo inserção	Tempo pesquisa
1 000						
5 000						
10 000						

Os tempos poderão ser medidos utilizando:

```
long inicio = System.nanoTime();

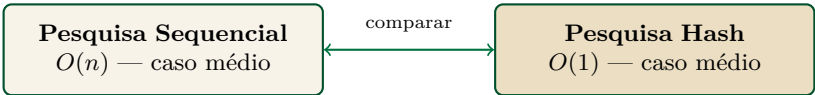
// operacao

long fim = System.nanoTime();

long tempo = fim - inicio;
```

13. Pesquisa Sequencial versus Hash

Deverá ser implementada uma pesquisa sequencial para comparação com a Tabela Hash.



Para evitar conclusões baseadas numa única execução, deverão ser efectuadas **várias pesquisas**, utilizando:

- ❖ códigos existentes;
- ❖ códigos inexistentes.

Deverá ser calculado o tempo médio e os resultados deverão ser interpretados no relatório.

14. Menu da Aplicação

A aplicação deverá possuir uma interface textual semelhante a:

```
=====
      SISTEMA DE GESTAO DE ESTUDANTES
=====

1 - Carregar ficheiro CSV
2 - Pesquisar estudante
3 - Inserir estudante
4 - Remover estudante
5 - Estatisticas dos estudantes
6 - Estatisticas da Hash Table
7 - Executar teste de desempenho
8 - Comparar pesquisa sequencial e Hash
0 - Sair

Opcao:
```

15. Relatório

O relatório deverá apresentar:

1. descrição da solução;
2. representação dos estudantes;
3. organização da Tabela Hash;
4. funções h_1 e h_2 ;
5. tratamento de colisões;
6. funcionamento de REMOVIDO;
7. redimensionamento;
8. estatísticas dos estudantes;
9. estatísticas da Hash Table;
10. resultados experimentais;
11. comparação entre pesquisa sequencial e Hash;
12. interpretação dos resultados;
13. conclusão.

Relação que deverá ser discutida

Factor de carga → Colisões → Sondagens → Desempenho

16. Entrega

Deverão ser entregues:

- ❖ código-fonte Java;
- ❖ relatório em PDF;
- ❖ resultados dos testes experimentais;
- ❖ instruções de execução da aplicação.

O ficheiro `estudantes.csv` fornecido pelo docente deverá ser utilizado durante os testes.

17. Cotação

Componente	Valores
Leitura e validação do CSV	2,0
Implementação da Tabela Hash	3,0
Double Hashing e tratamento de colisões	2,5
Inserção, pesquisa e remoção	2,0
Factor de carga e redimensionamento	2,0
Estatísticas dos estudantes	2,5
Estatísticas da Tabela Hash	1,5
Experiências e comparação de desempenho	2,0
Relatório e interpretação	1,5
Qualidade e organização do código	1,0
TOTAL	20,0

18. Defesa do Trabalho

Durante a defesa, poderão ser solicitadas operações como:

- ❖ explicar as funções Hash;
- ❖ demonstrar uma colisão;
- ❖ pesquisar um código existente;
- ❖ pesquisar um código inexistente;
- ❖ inserir um estudante;
- ❖ remover um estudante;
- ❖ demonstrar o funcionamento de REMOVIDO;
- ❖ explicar o redimensionamento;
- ❖ interpretar as estatísticas;
- ❖ efectuar pequenas alterações no código.

Teste obrigatório de compreensão

O estudante poderá ser solicitado a:

1. identificar duas chaves envolvidas numa colisão;
2. remover a primeira;
3. pesquisar a segunda;
4. demonstrar que a pesquisa continua através da posição marcada como REMOVIDO.

A operação deverá continuar a funcionar correctamente.